

👉 Video Cryptographie symétrique et asymétrique - Comprendre le SSL (21 min 19s)

👉 https://www.youtube.com/watch?v=jMIAoAVbcsw&list=PLYsJ-3MUn_eeYwSgJ3Z_hfrIzGqYOGAaj

cripto symétrique :

- 1 :50 < principe
- 3 :40 < ex de la scytale des spartiates et du chiffage de césar
- 6 :00 < ex contemporains (Vigenere, DES, triple DES, AES)
- 7 :30 < Les pour (rapidité) et les contres (beaucoup de clés : une clé par connexion = $\frac{n(n-1)}{2}$) du chiffage symétrique et nécessité de partager la clé (sans la compromettre) avant toute communication)

cripto asymétrique :

- 11 : 42 < principe (assure que le récepteur peut lire le msg s'il est émis avec sa clé publique et décoder avec sa clé privée, assure de l'identité de l'émetteur s'il est émis avec la clé privée de l'émetteur et déchiffre avec la clé publique de l'émetteur (signature numérique))
- 18 :15 < Les pour (deux clés par personne) et les contres (lenteur) du chiffage asymétrique

I. SECURISATION NECESSAIRE

1. RAPPEL SUR LE PROTOCOLE HTTP (HYPERTEXT TRANSFER PROTOCOL)

Lorsqu'un navigateur utilisé sur une machine client interprète un clic sur le lien hypertexte <http://lyceepapeclement.fr/>.

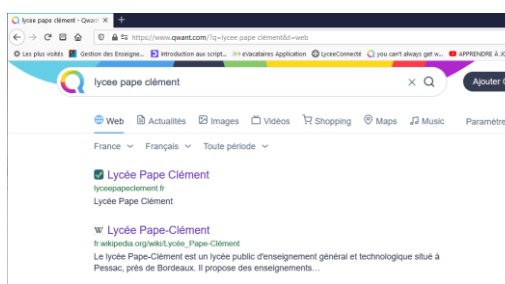
- Le navigateur décode l'URL du site <http://lyceepapeclement.fr/> (format URL : Uniform Ressources Locator) du serveur :
 - Le navigateur isole le protocole (ici, http) et le nom de domaine (ici, lyceepapeclement.fr) et le chemin vers la ressource demandée par l'utilisateur (ici, /, qui désigne la racine du site)
 - Un autre protocole (le protocole DNS) associe à l'adresse URL, l'adresse IP du serveur.
- Le navigateur établit alors une connexion TCP vers l'adresse IP du serveur web (sur le port 80)
- Une fois la connexion établie, client et serveur échangent alors les données en utilisant le protocole http (demandes de pages web (requêtes GET, retour des pages web du serveur, formulaires (requêtes POST,...))

Ordinateur Client

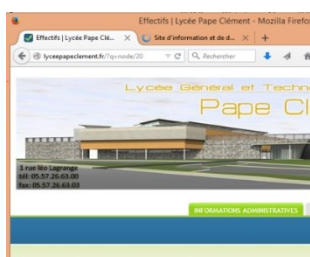
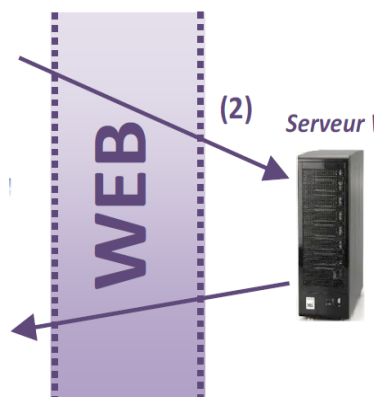


(1)

Navigateur Client



Serveur Hébergeur du site



(4) Page Web Affichée



(3) Code Page Web

2. RAPPEL SUR LES COUCHES INTERNET

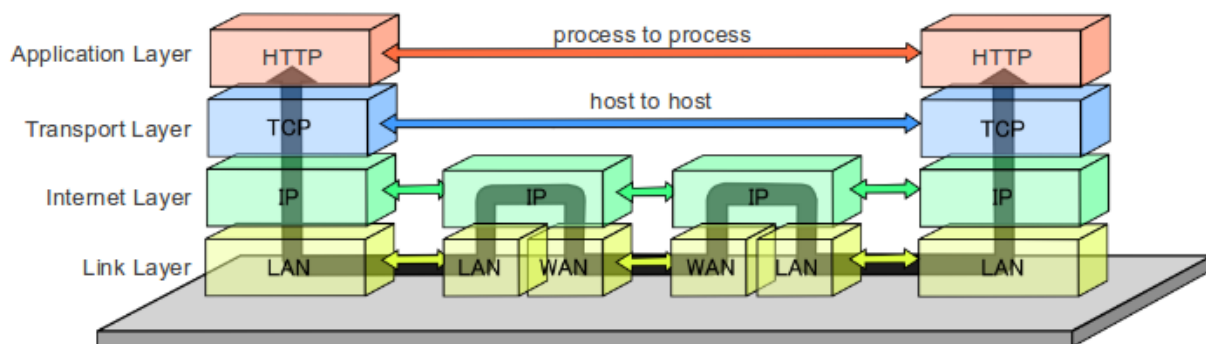
La transmission des données sur un réseau TCP/IP peut être décrite selon un modèle en 4 couches (voir le [modèle OSI](#)) dans lequel les protocoles TCP et IP jouent un rôle prédominant.

- ✚ Application (http, ftp, ...smtp/pop3)
- ✚ Transport (TCP pour fiabiliser la connexion, UDP pour l'adressage du port) :
- ✚ Internet (IP pour le routage)
- ✚ Accès réseau (liaison-physique ethernet, wifi, ...)

Chaque couche a pour fonction de traiter des données en provenance d'une couche adjacente et à destination de l'autre couche adjacente.

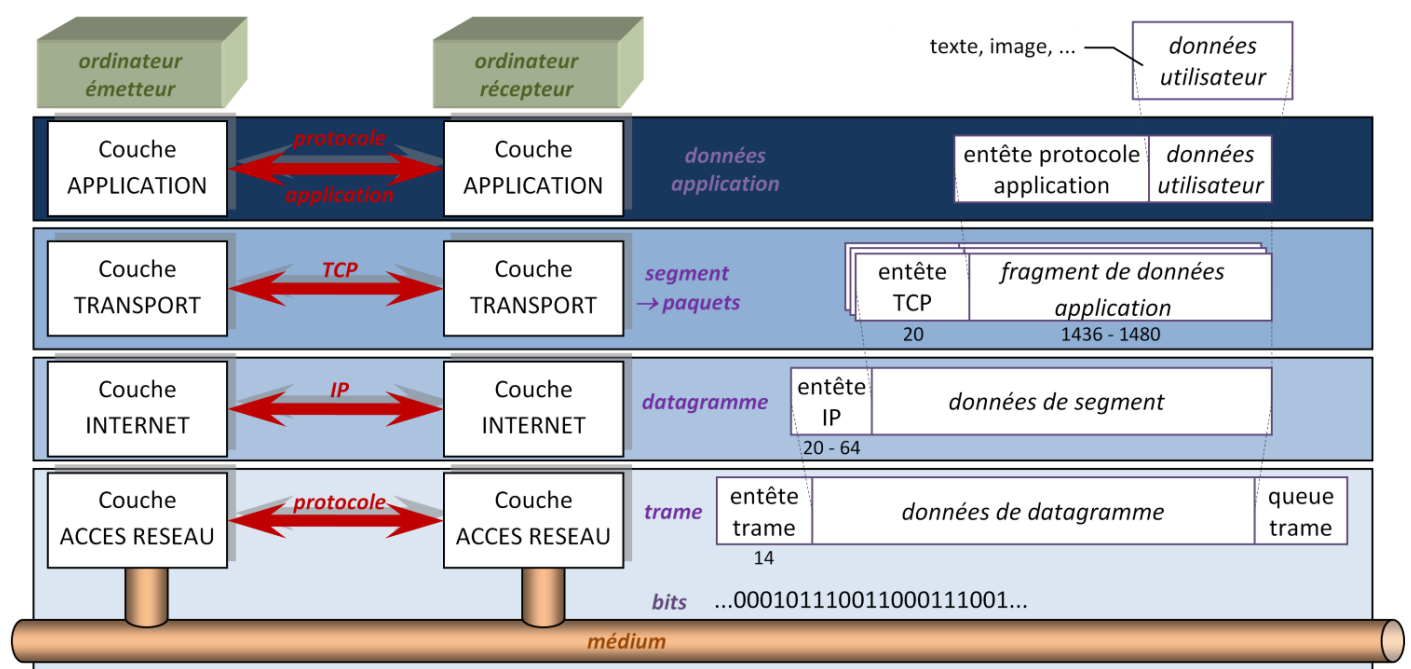
Ex 1 : Communication de deux applications http (navigateur sur un client avec un serveur hébergeur de site par exemple) entre deux hôtes connectés sur Internet par deux routeurs.

Data Flow of the Internet Protocol Suite



Sur les **médias** (supports physiques de transmission : câbles, fibres optiques, ondes électromagnétiques, ...) reliant physiquement les composants, les données circulent sous la forme de **trames**.

La **trame** se construit au passage des 4 couches du modèle, chaque couche **encapsulant** les données de la couche supérieure :



Les routeurs interviennent donc uniquement sur les paquets de la couche internet à acheminer. Ils n'ont pas à désencapsuler les couches supérieures. **Des routeurs malveillants peuvent cependant inspecter les paquets pour en connaître le contenu.**

3. LES PROBLÈMES À APPREHENDER ET LES DEUX SOLUTIONS EXISTANTES

Les échanges exigent la confidentialité des informations entre la source et la destination (quelle que soit la nature des échanges mais en particulier en cas de transactions bancaires, informations personnelles ou professionnelles, etc.).

Dans le cadre d'une communication TCP/IP, la sécurisation des échanges nécessite donc une réflexion autour des deux questions suivantes :

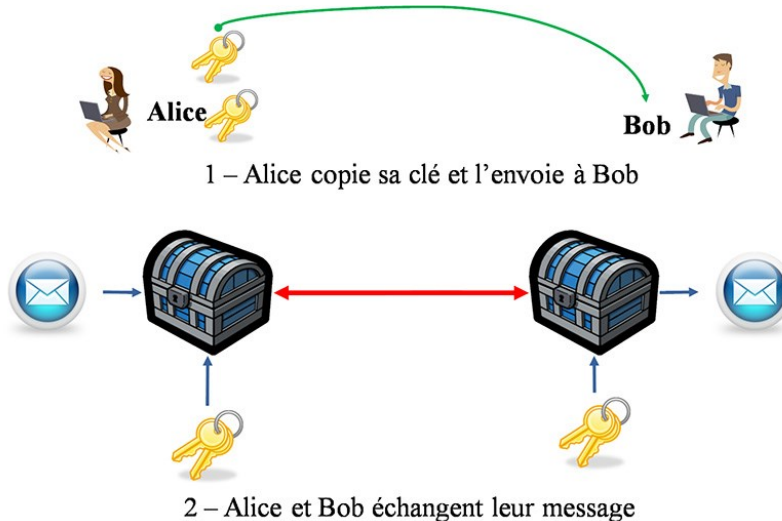
- ✚ Comment chiffrer le contenu des communications, afin qu'elles ne soient lisibles que par la source et la destination ?
- ✚ Comment garantir que le serveur auquel on se connecte est bien celui de la personne et/ou de l'entité à laquelle on pense se connecter ?

Les deux chiffrements utilisés sont les suivants :

- ✚ un chiffrement symétrique qui utilise une clé unique, connue seulement de l'émetteur et du récepteur. L'émetteur chiffre le message avec la clé et le récepteur le déchiffre avec la même clé.
- ✚ Par un chiffrement asymétrique qui utilise un couple de clé, l'une publique connue de tout le monde, l'autre privée connue uniquement par le récepteur. L'émetteur chiffre le message avec la clé publique. Le récepteur le déchiffre avec la clé privée.

II. LE CHIFFREMENT SYMETRIQUE

1. PRINCIPE



Dans un chiffrement symétrique, la clé utilisée par l'expéditeur pour chiffrer le message est la même que celle utilisée par le récepteur pour déchiffrer le message.

Des chiffrements Anciens :

Substitution monoalphabétique : César

Substitution polyalphabétique : Vigenère, [Enigma](#) (beaucoup plus récent).

Des chiffrements Récents :

- **Chiffrement par bloc** : système de chiffrement (symétrique) dans lequel le message clair est découpé en blocs d'une taille fixée, et chacun de ces blocs est chiffré. **DES et AES** sont les plus célèbres.
- **Chiffrement par flux** (ou par flot) : il permet de traiter des données de longueur quelconque, il n'a pas besoin de les découper. **Vernam, RC4, A5, E0** (utilisés dans les transmissions par ondes)

1. UN PREMIER EXEMPLE : LE CODAGE DE CESAR AVEC UNE CLE DE 13.

- En ne considérant que les lettres en majuscule et non accentuées, l'alphabet latin se réduit à 26 lettres (ABCDEFGHIJKLMNOPQRSTUVWXYZ)
Avec un décalage de 13 : (A → N) et (N → A). On utilise bien la même clé, il s'agit donc d'un chiffrement symétrique.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M

Par exemple le mot **python** est chiffré en **clguba** et **clguba** est déchiffré avec la même clé (13) pour donner **python**.

Exo 1.

- On donne
`d = {chr(i+ord('A')): chr((i+13)%26+ord('A')) for i in range(26)}.`
 Quel est le contenu de d ?
- Écrire une fonction **cesar_symetrique(s)** qui renvoie la chaine chiffrée par le chiffrement de César avec une clé de 13 sur la chaine **s** passée en paramètre. On pourra utiliser le dictionnaire **d**. Cette fonction code par décalage de 13 caractères la chaine **s** sans toucher les caractères d'espace.
- Vérifier que cette même fonction est bien celle d'un chiffrement symétrique

[chiffrement_symetrique.py](#)

2. UN SECOND EXEMPLE : UTILISATION DE XOR (CHIFFREMENT DE VERNAM)

- XOR est le « ou exclusif ». C'est ici une opération de logique bit à bit qui renvoie 0 si les deux bits sont égaux et 1 sinon.

Exo 2. Compléter :

```

0x7A xor 0xD3 = 0b01111010 xor 0b11010011 =
0xA9 xor 0x7A =
0xD3 xor 0xA9 =
xor 0xD3 = 0x00
0x7A xor = 0xFF
132 xor 28 =
0 xor a =
0xFF xor a =
    
```

Exo 3. La console Python indique :

```

>>> ord('b')
98
>>> hex(ord('b'))
'0x62'
    
```

Le code Unicode de la lettre 'b' est 98 ou 0x62.

1.

a. Ecrire une fonction **unicode(s)** qui retourne un triplet constitué de

- ✚ la liste des codages unicode de chaque caractère de la chaîne **s**, exprimé en décimal,
- ✚ la liste des codages unicode de chaque caractère de la chaîne **s**, exprimé en binaire,
- ✚ la liste des codages unicode de chaque caractère de la chaîne **s**, exprimé en hexadécimal.

b. Compléter les tableaux suivants qui donnent les codages Unicode du message à chiffrer « logiciel » et du mot « nsi » qui va être utilisé comme clé de chiffrement :

msg	caractères	l	o	g	i				
	unicode	108	111						
	decimal								
	unicode	6C	6F			63	69		
	hexa								
	unicode	1101100	1101111					1100101	1101100
	binaire								

clé	caractères	n	s	i	n	s	i	n	s	i
	unicode	110	115	105	110	115	105	110	115	
	decimal									
	unicode									
	hexa									
	unicode									
	binaire									

c. La méthode de chiffrement consiste ici à aligner le mot et la clé en la répétant autant de fois que nécessaire et d'effectuer **un XOR entre les codes Unicode du message et de la clé**. Compléter le tableau :

msg chiffré	caractères								
	unicode	2	28	14	7				
	decimal								
	unicode	2	1C	E	7				
	hexa								
	unicode	10	11100	1110	111				
	binaire								


[chiffrement.xlsx](#)

d. Peut-on donner le message chiffré sous forme de caractères ?

2. Ecrire en Python la fonction suivante :

```
def chiffre_xor(msg, cle):
    """prend les chaines msg et clé en paramètres et
    renvoie la liste des octets en décimal du message chiffré """
```



En Python l'opérateur `^` permet d'effectuer un **xor** directement sur deux entiers :

```
>>> bin(1)
'0b1'
>>> bin(3)
'0b11'
>>> 1^3
2
```

Résultat de test attendu :

```
>>> chiffre_xor("logiciel", "nsi")
unicode (binaire) de 'logiciel' : ['0b1101100', '0b1101111', '0b1100111', '0b1101001',
'0b1100011', '0b1101001', '0b1100101', '0b1101100']
unicode (hexa) de 'logiciel': ['0x6c', '0x6f', '0x67', '0x69', '0x63', '0x69', '0x65', '0x6c']
unicode (decimal) de 'nsi' : [110, 115, 105]
unicode (binaire) de 'nsi' : ['0b1101110', '0b1110011', '0b1101001']
unicode (hexa) de 'nsi' : ['0x6e', '0x73', '0x69']
message chiffré (en decimal): [2, 28, 14, 7, 16, 0, 11, 31]
```

1.

- Montrer que : Si $b_1 \text{ xor } b_2 = b_3$ alors $b_1 \text{ xor } b_3 = b_2$ et $b_2 \text{ xor } b_3 = b_1$
- Justifier que le chiffrement est symétrique (c'est-à-dire que la même procédure que le codage permettra de décoder avec la même clé.).
- Écrire une fonction qui déchiffre :

```
def déchiffre_xor(msg_decimal, cle):
    """prend les chaines msg_decimal (liste des entiers codages
    unicode du msg chiffré) et clé en paramètres et renvoie la chaîne
    du message déchiffré """
```



[chiffrement_symetrique.py](#)

Rq1.

Ce type de chiffrement à base binaire peut être utilisé pour chiffrer n'importe quelle donnée informatique, un texte, une image, une vidéo, etc. et si la clé respecte 3 conditions, il a été démontré que ce chiffrement offre une sécurité théorique absolue. Pour cela la clé doit :

- être une suite de caractères au moins aussi longue que le message à chiffrer.
- Ses caractères doivent être choisis de façon totalement aléatoire.
- Chaque clé ne doit être utilisée qu'une seule fois (on appelle aussi ce chiffrement le chiffrement à masque jetable).

Dans ce cas, ce chiffrement est dit "parfait" ...mais sa mise en œuvre est impossible, puisque que tout le problème est l'échange de la clé. Pour chiffrer "parfaitement" un message de n caractères il faut une clé de n caractères, clé qui doit être échangée en secret entre Alice et Bob, donc dans ce cas, autant échanger directement le message de n caractères au lieu d'échanger la clé de n caractères...

Aujourd'hui dans le cas de transmissions sur Internet, le chiffrement n'est pas parfait (on parle de **chiffrement pratiquement sûr**, c'est à dire qu'un message chiffré ne peut pas être décrypté en temps « humainement raisonnable ») mais il utilise des clés plus courtes que le message à chiffrer et réutilisent plusieurs fois la même clé.



Chiffrement symétrique: (10 :57) <https://www.youtube.com/watch?v=pxRU8vgzyq4>

- Vernam (xor, chiffrement parfait mais théorique,)
- Chiffrement par bloc, chiffrement par flux
- Importance des grands nombres pour la sécurité du chiffrement
- Chiffrement par bloc : modes d'opérations :
 - ECB (Electronic Code Book)
 - CBC (Cipher **B**lock **C**hain)
- Chiffrement par flux (à 9 :30) : modes d'opérations :
- A 10 : 15 : le problème de l'échange des clés

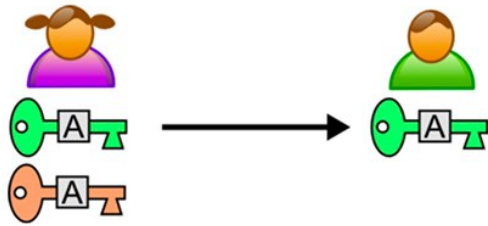
III. LE CHIFFREMENT ASYMETRIQUE

3. PRINCIPE

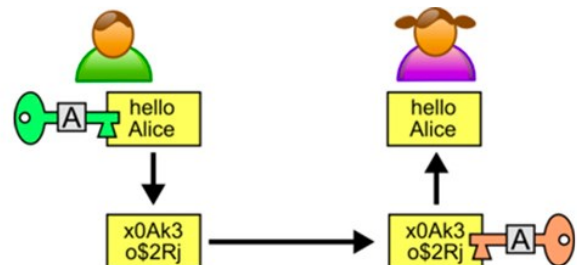
Alice veut recevoir un message chiffré de Bob.

Alice génère 2 clés : la clé publique (la clé verte) qu'elle envoie à Bob et la clé privée (la clé orange) qu'elle conserve secrète.

Bob chiffre le message qu'il veut envoyer à Alice avec la clé publique d'Alice (qui ne sert qu'à chiffrer) et envoie le texte ainsi chiffré à Alice. Alice reçoit le texte chiffré et elle seule peut le déchiffrer avec sa clé privée.



La cryptographie asymétrique : Première Étape



La cryptographie asymétrique : Deuxième et Troisième Étape

1. CESAR ASYMETRIQUE

Exo 4. [.chiffrement_asymetrique.py](#)

- Dans l'exemple précédent du code Cesar, expliquer pourquoi la clé 13 garantit-elle un chiffrement symétrique ? D'autres clés peuvent-elles garantir elles aussi la symétrie du chiffrement ?
- Le principe du Code César peut-il être utilisé dans le cadre d'un chiffrement asymétrique ? si oui, dans quelles conditions ?
- Ecrire les deux fonctions suivantes dont on donne les *docstrings* :

```
def chiffre_cesar(msg, clé_chiffrement):
    """prend en parametre le message à chiffrer (chaine)
    et la clé de chiffrement,
    renvoie la chaine du message chiffré"""

def dechiffre_cesar(msg, clé_chiffrement):
    """prend en parametre le message à déchiffrer (chaine) et
    la clé utilisée pour le chiffrement,
    renvoie la chaine du message en clair"""
```
- Pourquoi ce chiffrement n'est-il pas un chiffrement asymétrique très rassurant quant à la sécurité des échanges ?

IV. COMPLEMENT - LE SYSTEME RSA (SIMPLIFIE)

Le système RSA (pour Ron Rivest – Adi Shamir – Leonard Adleman) date de 1978.



Chiffrement asymétrique (! attention nombres belges, ou suisses (voir à 12 :15) !!) : <https://www.youtube.com/watch?v=MUNyEoU5tSo>

- 0 :30 Principe des clés publique et privée
- 0 :54 : un exemple d'échange de clés publiques, puis échanges de données
- 2 :55 : rappel sur le modulo
- RSA à 3 :30 à base de factorisation par deux nombres premiers : la découverte des deux facteurs pour un nombre suffisamment grand est supposée « difficile » et en tout cas impossible **rapidement**. Tout repose sur cette supposition et sur l'adverbe « rapidement ».
- ELGamal à 6 :30 à base de log discret (! hors programme NSI)
- Echange de clé Diffie-Hellman à 7 :10

1. PRINCIPE**Choix de la clé publique**

L'émetteur, Alice, choisit deux nombres premiers p et q distincts et supérieurs ou égaux à 3, et e premier avec le produit $f = (p-1)(q-1)$ (e n'a pas de diviseurs communs avec f) et tel que $0 < e < (p-1)(q-1)$. On note

$$n = p \times q$$

Le couple (e, n) est alors la clé publique d'Alice (c'est-à-dire que tout le monde peut se la procurer)

Ex 2 : $n = 377$ est le produit des deux nombres premiers ($377 = 13 \times 29$).
 On considère alors l'entier $f = (13 - 1) \times (29 - 1) = 12 \times 28 = 336$ et
 on choisit $e = 5$ un nombre premier inférieur à f .
 la clé publique choisie est alors : $(e; n) = (5; 377)$

Exo 5. Méthode de chiffrement : ([chiffrement.xlsx](#))

Tout le monde peut envoyer un message à Alice en utilisant sa clé publique (e, n) de la manière suivante :

1. Pour coder un caractère car ,
 - ✚ On détermine le code Unicode c du caractère car .
 - ✚ le reste de la division Euclidienne de c^e par n est alors le caractère chiffré c_rsa
- a. Déterminer le résultat du chiffrement de 'b'.
- b. Déterminer le résultat du chiffrement de 'n'.

2. Pour décoder un caractère chiffré :

La clé privée

d est l'**unique** entier, uniquement connu d'Alice, tel que $1 \leq d < f$ et tel que le reste de la division euclidienne de $e \times d$ par f est 1. (avec $f = (p-1)(q-1)$).

Le couple $(d; n)$ est alors la clé privée d'Alice.

Si c_rsa est le caractère chiffré à déchiffrer, le reste de la division euclidienne de $(c_rsa)^d$ par $n = pq$ est alors le code Unicode du caractère en clair.

- a. Justifier que $d = 269$ convient.
- b. Déterminer le caractère en clair chiffré en 265.
- c. Déchiffrer le message « 265 118 316 316 297 ».

3. Le chiffrement dans ces conditions paraît-il sécurisé ?

Rq2. la sécurité engendrée par un tel système réside dans la nécessité d'un temps raisonnablement long pour retrouver la clé privée par le calcul : elle nécessite donc l'utilisation de nombres premiers très grands. Certains de ces nombres, trop célèbres, sont aussi à éviter (par exemple, les nombres de Mersenne de la forme $2^p - 1$ où p est lui-même premier.)

Une clé publique (un peu plus) efficace peut être formée avec $p = 9967$ et $q = 9973$, soit $n = 9967 \times 9973 = 99400891$

Exo 6.

1. Ecrire une fonction **est_premier(n)** qui renvoie un booléen précisant si **n** est premier ou non.
2. Ecrire une fonction **facteurs_premiers(n)** qui renvoie la liste des facteurs premiers de la décomposition de **n**. Par exemple : **facteurs_premiers(300)** renvoie **[2,2,3,5,5]**
3. Ecrire une fonction **factorisation_RSA(n)** qui renvoie les deux facteurs premiers p et q , s'ils existent tels que $n = p \times q$, et qui renvoie **None** si ces facteurs premiers n'existent pas.
Tester cette fonction et mesurer son temps d'exécution sur les entiers n suivants :
377, 437, 99400891, puis 99999640000243
4. Ecrire une fonction **premiers_entre_eux(a,b)** qui renvoie **True** si a et b n'ont pas d'autres diviseurs communs que 1, **False** sinon.
5. Ecrire une fonction **clés_valides(n,e,d)** qui renvoie **True** si (e,n) est une clé publique valide et (d,n) est une clé valide, et **False** sinon.
6. Tester sur les clés suivantes :
 - a. clé publique $(e;n)=(5;377)$; clé privée $(d;n) = (269,377)$
 - b. clé publique $(e;n)=(41;437)$; clé privée $(d;n) = (29, 437)$
 - c. clé publique $(103, 697)$; clé privée $(87, 697)$
7. Écrire les deux fonctions (l'une qui chiffre, l'autre qui déchiffre un message) relatives au système RSA.
Tester avec les différents couples de clés de la question précédente :

```
def chiffre_RSA(msg,cle_n,cle_e):
```

```
    """prend en parametre le message à chiffrer (chaîne) et la clé de chiffrement
    renvoie la chaîne du message chiffré"""
```

```
def dechiffre_RSA(msg,cle_n,cle_d):
```

```
    """prend la liste msg_decimal (liste des entiers codages unicode du msg chiffré)
    et la clé en paramètres et renvoie la chaîne du message déchiffré """
```

8. Tester vos fonctions de chiffrement RSA avec votre voisin(e), en ayant au préalable diffusé vos clés publiques :
chiffrez votre message avec sa clé publique. Envoyez le msg chiffré (par mail, par ex.). Il le déchiffre avec sa clé privée.



[chiffrement_RSA.py](#)

V. PROTOCOLE SECURISE



https://www.youtube.com/watch?v=7W7WPMX7arI&feature=emb_logo

- Chiffrement symétrique, asymétrique,
- Transmission de la clé publique : le problème de l'homme du milieu (à 5:15)
- Pour certifier l'identité du porteur de la clé publique transmise : le certificat (à 6 :30)
- Protocole https (à 8 :00)

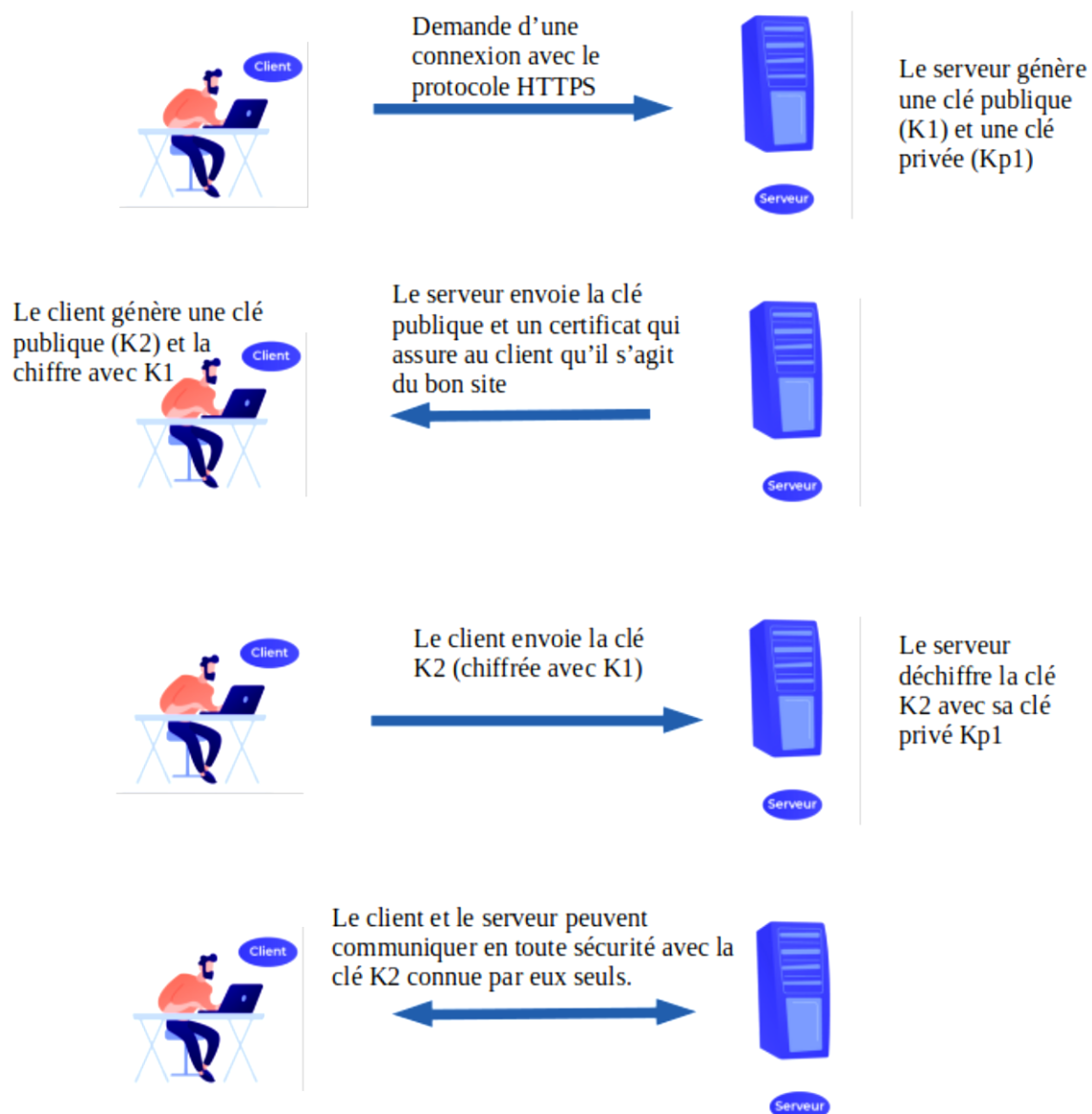
HTTPS est la version sécurisée de http. Elle en corrige les 2 principales lacunes :

- ✚ Assure le chiffrement des données grâce à une clé de cryptage asymétrique.
- ✚ Assure l'identité d'un serveur détenteur du certificat TLS correspondant. Cette authentification est réalisée par l'utilisation d'un certificat numérique délivré par une autorité de certification (AC).

HTTPS utilise le protocole TLS (Transport Layer Security 1999, dernière version en 2018) anciennement connu sous le nom de SSL (Secure Sockets Layer créé en 1994 par Netscape), on parle d'ailleurs parfois de SSL/TLS.

✚ Les deux méthodes de chiffrement **symétrique** et **asymétrique** sont utilisées lorsqu'un navigateur demande une page web avec le protocole HTTPS.

Dans le schéma ci-dessous, on a représenté les échanges de communications entre un client et un serveur avec le protocole HTTPS.



✚ Les échanges entre client et serveur se font donc par chiffrement **symétrique** avec une clé K2 après que celle-ci a été échangée avec un chiffrement **asymétrique**.

Le protocole https organise l'échange d'une clé de chiffrement symétrique à l'aide du chiffrement asymétrique.